```
Void intercalar(tCola &q){
    int n = q.size()/2;
    tPila p;

    for(int i=0; i<n; i++){
        p.push(q.frontValue());
        q.pop();
    };

    for(int i=0; i<n; i++){
        q.push(p.topValue());
        p.pop();
    };

    for(int i=0; i<n; i++){
        int dato = q.frontValue();
        q.pop();
        q.push(dato);
    };
    for(int i=0; i<n; i++){
        p.push(q.frontValue());
        q.pop();
    };
    for(int i=0; i<n; i++){
        q.push(p.topValue());
        p.pop();
        int dato = q.frontValue();
        q.pop();
        q.push(dato);
    };
};
```

Ejercicio A

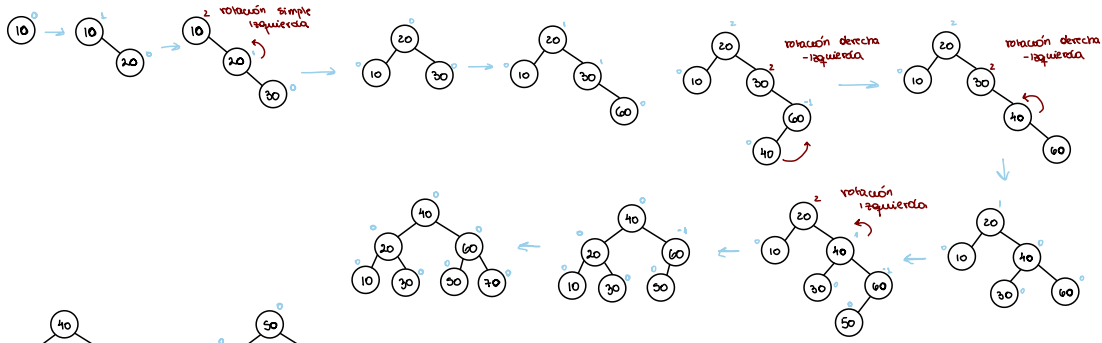
```
Void preOrden(tNodoArbolBin *raiz, tPila &camino, string accion, tipoElem &hoja,
              int &minimo){
    if(raiz == NULL) return;
    camino.push(raiz);
    bool esHoja = raiz->izq == nullptr && raiz->der == nullptr;
    if(esHoja && raiz->accion == accion){
        int dist = camino.size() - 1;
        if(dist < minimo){
            minimo = dist;
            hoja = raiz->info;
        };
    };

    preOrden(raiz->izq, camino, accion, hoja, minimo);
    preOrden(raiz->der, camino, accion, hoja, minimo);
    camino.pop();
};

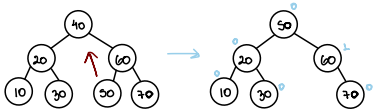
tipoElem decisionesMinimas Help(tNodoArbolBin *raiz, tPila &camino, string accion,
                                tipoElem &hoja, int &minimo){
    hoja = -1;
    minimo = INT_MAX
    preOrden(raiz, camino, accion, hoja, minimo);
    return hoja;
};
```

Ejercicio B

a)



b)



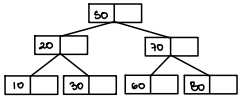
c) 50 - 20 - 10 - 30 - 60 - 70

d)

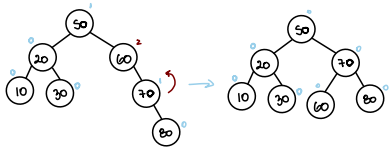


e)

I) El árbol 2-3 sí aumenta su altura:



II) El AVL no aumenta su altura:



III) Si continuaran llegando personas la mejor estructura sería el árbol 2-3 puesto que puede almacenar un gran volumen de datos de forma más eficiente que AVL.

Ejercicio C